MCNF Algorithms

Assumptions

1. Integrality (supplies and demands are integer) thus by unimodularity $\exists$ optimal integer solution
2. l_{ij}, no parallel arcs, no symmetric arcs,
3. $\exists$ feasible solution
4. $\forall (i,j) \in E, c_{ij} \geq 0$ (restrictive for negative costs and infinite upperbound)
5. $\forall i,j \in V, \exists$ path $i \to j$ of ∞ capacity

   i,j both point to an exchange vertex X which points back to i and j (through a temp vertex to prevent symmetric arcs) all with infinite capacity. Suppose all arcs have a cost of $BC+1$ see below

   This makes it so that each node always has a path to every other node but note that some paths might require the use of the exchange vertex resulting in an infeasible solution in the original problem.

Question Max cost of a MCNF? $\exists$ feasible solution

- wlog no cycles bc they just increase the cost
- wlog integer solution

For paths, there must be paths from all source nodes to all sink nodes. Thus we can write $B = \sum_{i:b_i>0} b_i$ and $C = \sum_{(i,j)\in E} c_{ij}$ so cost of MCNF $\leq BC$ as an extreme worst case scenario.

Primal Dual Method

Idea is find x, π and slowly reduce the infeasibility region (start with a not necessarily feasible solution in primal and iterate progressively)

Assumptions from above

Invariant:

- π feasible for dual
- x solution for primal
- x, π complementary slackness

At every iteration, reduce x infeasibility

Def A pseudo-flow x satisfies capcity constraints ($x_{ij} \leq u_{ij}$) but not necessarily mass-balance constraints.

$e_i = b_i - \sum_{j:(i,j)\in E} x_{ij} + \sum_{i:(i,j)\in E} x_{ij}$

excess at $i \in V$. Note it is feasible if $e_i = 0$ for all i

> Thus we start our algorithm with a feasible flow of $x^0 = 0$ so $\pi^0 = 0$ because $G(x^0) = G(x)$ is valid for $c_{ij}^{\pi 0} \geq 0$ where $G(x^0)$ is the residual network flow for our flows x^0.
>
> > Recall that $c_{ij}^{\pi} = c_{ij} - \pi_i + \pi_j$

Lemma For $\pi^i \to \pi^{i+1}$

Let x be a pseudoflow, π feasible node potentials, $s \in V$, $c_{ij}^{\pi} \geq 0, \forall (i,j) \in E(x)$

> d_i = length of shortest path from s to i in $G(x)$ w/ length c_{ij}^{π}

Then, $\pi' = \pi - d, c_{ij}^{\pi'} \geq 0, \forall (i,j) \in E(x)$ and actually $c_{ij}^{\pi'}$ is exactly 0 along shortest paths

> In other words, we use the shortest paths algorithms and x^i, π^i to find π^{i+1}

Proof Bellmin Ford $d_j \leq d_i + c_{ij}^{\pi}, (j \in V, (i,j) \in E(X))$

> $c_{ij}^{\pi'} = c_{ij} - \pi_i' + \pi_j' = c_{ij} - \pi_i + i + \pi_j - d_j = c_{ij}^{\pi} + d_i - d_j$
>
> By the Bellmin ford conditions: $c_{ij}^{\pi} + d_i - d_j \geq c_{ij}^{\pi} - c_{ij}^{\pi} = 0$
>
> The Bellmin Ford inequality is 0 along shortest paths, thus $c_{ij}^{\pi'} = 0$ along shortest paths

Lemma For $x^i \to x^{i+1}$

Let x be a pseudoflow, π feasible node potentials, $s \in V$, $c_{ij}^{\pi} \geq 0, \forall (i,j) \in E(x)$

> $x' = x$+flow $s \to i$ along the shortest path $s \to i$
>
> From the previous lemma, $c_{ij}^{\pi} = 0$ along the shortest paths from $s \to i$ in the residual network
>
> $c_{ij}^{\pi} \geq 0, \forall (i,j) \in E(x')$

In other words we reduce the excess e for s and i at each iteration.

Proof We have an original $G(x)$ and we create a $G(x')$ with the only changes along the shortest path from s to i.

Since we are only increasing flow along this path, the entire rest of the residual network stays the same. Along the path however, we could add new arcs in reverse and potential remove forward arcs (if we reach capacity).

Since $c_{ij}^{\pi} = -c_{ji}^{\pi}$ the reversed arcs are still 0 and we know the forward reduced costs stay the same since we haven't changed the π's.

Successive Shortest Paths Algorithm

for MCNF

Transform instace so that assumptions above hold

- $e \leftarrow b$ //Initial Excess to supply
- $x \leftarrow 0$ //Initial flow $= 0$
- $\pi \leftarrow 0$ //Initial potentials $= 0$
- $X \leftarrow \{i \in V : e_i > 0\}$ // Initial excess nodes
- $D \leftarrow \{i \in V : e_i < 0\}$ // Initial deficit nodes

```
while X is not empty:
    choose s in X, t in D
    compute shortest path from s to i in V
        in G(x) with length c_{ij}^\pi
        and d_i # shortest distance s to i

    pi -= d
    delta = min(e_s, -e_t, u_{ij}: (i,j) in shortest path from s to t)
    x += delta # (flow along shortest path s to t)
    update G(x),e,X,D
```

hint as we run the algorithm postponse creation of interchange vertex until needed

hint $c_{ij}^{\pi} = 0$ along shortest paths

hint $c_{ij}^{\pi} = -c_{ji}^{\pi}$

Runtime

Suppose S is the time for the shortest paths

Thus runtime is $O(\#iter \times O(S))$

We can work on calculating num iterations as $B = \sum_{bi>0} b_i$ because we now at each iteration we send at least one unit of flow.

Thus the run time is $O(BS)$

> This is not polynomial because if we increase the number of bits by 1 we double the run time.

Optimality Proof idea

We end up with reduced costs that are always greater than 0 and we know that this is feasible

Runtime improvements

Idea 1: Choose s,t so that $\max \delta$ - Tractability is - Tradeoff, $\max \delta$ but now we need to compute the max

some success but nothing crazy

Idea 2: find δ that is sufficiently large: $\delta \geq \Delta$ - Capacity Scaling: $U = \max b_i$ - $\delta \geq \Delta = U, U/2, U/4, ..., 1$

rough outline:

```
for Delta = U, U/2, U/4,..., do:
    SSP for delta >= Delta
```

Capacity Scaling

Reminder of MCNF: SSP

Augments a flow $\delta \geq 1$ from a supply node to a demand node.

Remember the optimization of $\delta \geq \Delta = B, B/2, B/4, ...$

Δ - residual network:

> $G(x, \Delta)$: only edges w/ capacity $\geq \Delta$
>
> $X(\Delta) = \{i \in V, e_i \geq \Delta\}$

$$D(\Delta) = \{i \in V, e_i \leq -\Delta\}$$

Idea is generate the residual network graph and send Δ units of flow from s, t. Note that this will initially potentially make an unfeasible flow using an exchange vertex.

Induce the flow and calculate the reduced costs. Once we have the reversed costs, we can reverse the arcs of all negative arcs. The main problem in reversing the negative costs is for Arcs that we created by inducing the flow the upperbounds for these arcs are strictly less than Δ which is non infinite so we know we can do cost reversal.

Note that this now creates new supply and demand nodes thus needing the algorithm to run again.

```
for Delta = B down to 1 do:
    SSP on G(x,Delta) with delta=Delta
    Delta = Delta/2
    G(x,2)
```

Runtime

Time per augmentation $\times$ num of augmentations per scaling phase $\times$ num scaling phase

Time per augmentation is $O(S)$ because same shortest paths algorithm

Number of scaling phases is $\log(B)$

Let β denote the $\max b_i$, note that $U = \max u_{ij} < \Delta$

This means that num augmentations per scaling phase is $= \frac{totalsupply}{\Delta} = \frac{n\beta + m\Delta}{\Delta} \leq n + m$

Thus the total runtime is $O((n+m)S \log B)$. This is polynomial time!